CLI Options in the Wild

Rapport de projet

Introduction

Ce rapport présente le projet **CLI Options in the Wild**, réalisé dans le cadre d'un projet de recherche et développement encadré par Xhevahire Tërnava à Télécom Paris. Ce projet vise principalement à concevoir un outil de comptage des options proposées par les interfaces en ligne de commande. En s'appuyant sur ce script, l'étude cherche à analyser la progression du nombre d'options au fil des versions, afin de déterminer si ces outils deviennent plus complexes avec le temps.

La question de recherche peut être formulée simplement :

Les programmes en ligne de commande ont-ils de plus en plus d'options d'une version à l'autre ?

Les résultats de ce projet et les outils qui ont été développés sont mis à disposition via une page web statique.

Partie 1 : Le script compteur d'options CLI

Objectif

La première brique du projet est le développement d'un outil sur Python capable d'analyser n'importe quel programme installé sur une machine Unix et d'en compter les options disponibles. Ce compteur doit être générique : il ne doit pas être limité à un seul programme, mais fonctionner sur un large éventail d'outils (GNU Coreutils, Toybox, FFmpeg, etc.).

Fonctionnement

Le principe du compteur repose sur trois étapes :

1. Appel de l'aide du programme. Le script exécute le programme cible en lui passant un argument d'aide (`--help`, `-h`, `-usage`, ou d'autres variantes selon le programme). La sortie standard est capturée.

2. Extraction par expression régulière. Le texte de l'aide est analysé à l'aide d'une expression régulière qui détecte les motifs caractéristiques des options CLI :
- Les options longues (`--option-name`)
- Les options courtes (`-x`)

L'expression utilisée est : `--[a-zA-Z0-9][a-zA-Z0-9._-]*|-[a-zA-Z0-9]+`

3. Dédoublonnage et comptage. Toutes les occurrences trouvées sont placées dans une liste Python, puis nous éliminons les doublons pour retourner le nombre total d'options uniques.

Résultats

L'outil de comptage a été exécuté directement sur nos machines sur 335 programmes courants. Les résultats sont compilés dans un fichier CSV : " **latest.csv** " indiquant, pour chaque programme, le nombre d'options détectées dans la version actuellement installée.

Difficultés rencontrées

Hétérogénéité des formats d'aide : La principale difficulté a été la grande variabilité du format de l'aide entre les programmes. Si la très grande majorité utilise `--help`, certains outils utilisent `-h`, `-h all`, `--help full` (comme `curl`), voire une commande dédiée. Cette diversité a rendu le développement d'une approche universelle très complexe : il a fallu itérer plusieurs fois sur la logique de détection.

Faux positifs dans l'extraction : L'expression régulière peut extraire des séquences qui ressemblent à des options mais n'en sont pas (par exemple, des chemins de fichiers du type `-rw-r--r--`, ou des valeurs numériques). Des itérations successives sur le regex ont été nécessaires pour améliorer la précision sans trop restreindre le rappel.

Programmes indisponibles : Certains programmes ne sont pas installés sur toutes les machines. La gestion des timeouts et des codes de retour non nuls a nécessité une attention particulière pour éviter que le script ne se bloque indéfiniment.

Partie 2 : Build des anciennes versions et données historiques

Objectif

La seconde partie du projet vise à suivre l'évolution temporelle du nombre d'options. Pour cela, il est nécessaire d'obtenir des mesures pour de nombreuses versions successives d'un même programme et non seulement pour la version actuelle. Cela implique de compiler ou récupérer des binaires correspondant à des versions publiées au fil des années.

Fonctionnement

Deux approches ont été développées selon le programme ciblé :

Pour **GNU Coreutils et FFmpeg**, nous avons cloné le dépôt Git officiel, identifié le tag Git correspondant à la version la plus récente pour chaque année depuis 2005, puis compilé chaque version sur nos machines. Le script `**version_download.py**` automatise le clonage, la sélection d'un tag par année, et l'exécution des commandes de compilation et de test pour

chaque version. Une fois les binaires générés, notre script de comptage d'options leur est appliqué et les résultats sont écrits dans des fichiers CSV.

Pour **Toybox**, nous avons utilisé une méthode plus directe. Au lieu de lancer chaque programme pour essayer de lire son aide, nous avons 'ouvert le capot' du fichier : comme nous savions où les options étaient rangées dans le code interne du programme (dans une liste appelée toy_list), nous avons créé un script Python capable d'extraire ces informations directement depuis le fichier. Cela nous a permis de récupérer les données bien plus rapidement, sans même avoir besoin de lancer les logiciels.

Au total, les données couvrent :

- **GNU Coreutils et FFmpeg** : plus de 100 commandes sur une vingtaine d'années (2005–2026)
- **Toybox** : plus de 200 commandes sur une quinzaine de versions (2007–2025)

Résultats

Les données historiques sont regroupées dans un ensemble de fichiers CSV (un par couple programme/suite). Parmi les enseignements les plus marquants :

- Les commandes `sort`, `date`, `grep` et `tar` ont connu les augmentations les plus fortes (+10 à +24 options selon les implémentations).

- Des commandes classiques comme `cp`, `mv`, `chmod` ou `sed` ont également vu leur interface s'enrichir progressivement.

- La commande `ls` est intéressante car elle est présente dans deux suites (Coreutils et Toybox), ce qui permet de comparer l'évolution de deux implémentations différentes d'une même commande.

Nous avons, par la suite, rassemblé toutes ces datas dans un seul et même fichier csv : “**history.csv**”

Difficultés rencontrées

Compilation des anciennes versions : Essayer de faire fonctionner de vieux logiciels sur des ordinateurs récents a été un vrai défi. Le code source était parfois si ancien qu'il ne comprenait plus les outils de programmation modernes. Pour les versions les plus vieilles (avant 2010), nous avons dû modifier manuellement le code pour qu'il accepte d'être 'construit' sur nos systèmes actuels.

Dépendances manquantes : Certains programmes avaient besoin de petits composants externes (des bibliothèques) qui ne sont plus utilisés aujourd'hui et qui manquaient sur nos machines. Pour contourner ce problème, nous avons créé des 'remplaçants' simplifiés (des

fichiers de contournement) qui simulent la présence de ces composants, permettant ainsi aux anciens programmes de s'exécuter correctement.

Lecture directe des fichiers pour Toybox : Extraire les options directement depuis le fichier du programme, au lieu de lire son menu d'aide (--help), s'est avéré complexe. Nous avons dû décortiquer la structure interne du logiciel pour trouver où étaient cachées les listes d'options. Bien que cette méthode soit beaucoup plus fiable que l'analyse classique, elle a nécessité un sérieux travail de recherche pour comprendre le fonctionnement interne de Toybox.

Présentation du projet — La page web statique

Afin de rendre nos résultats accessibles, nous avons développé une page web statique regroupant toutes les dimensions du projet :

- Section « Téléchargement » : mise à disposition d'un fichier ZIP contenant le script compteur, accompagné d'explications exhaustives pour savoir comment l'installer et l'exécuter sur sa propre machine. Cet outil permet à n'importe quel utilisateur de mesurer le nombre d'options des programmes installés chez lui. Nous mettons également à disposition nos deux fichiers csv finaux : **Latest.csv et History.csv.**

- Outil de recherche : un outil de consultation permettant de sélectionner un programme et une implémentation (GNU Coreutils, Toybox) pour afficher le nombre d'options d'une version donnée, directement depuis notre base de données CSV.

- Section « Présentation et Résultats » : visualisations graphiques de l’évolution du nombre d'options et comparaison du nombre d’options entre différents programmes. Ces graphes sont issus des données statiques que l’on peut retrouver dans les deux csv : **Latest.csv et History.csv.**

Bibliographie

Contributeur/Source	Ressources et Liens
Yoan	- **Documentation officielle de Tkinter pour Python** (https://docs.python.org/fr/3/library/tkinter.html) - **Guide Matplotlib sur l'intégration de graphiques dans des fenêtres Tkinter** (https://matplotlib.org/stable/gallery/user_interfaces/embedding_in_tk_sgskip.html).
Baptiste	- **Ressources sur les modules Python *re*** (https://docs.python.org/3/library/re.html) - **Subprocess** (https://docs.python.org/3/library/subprocess.html) - **Analyses sur les standards d'aide CLI** (https://clig.dev/#help).
Théophile	- **Tutoriels pour la génération de diagrammes à barres avec Matplotlib via Python-Simple** (https://www.python-simple.com/python-matplotlib/barplot.php).
Ressources communes	- **Étude de Dan Luu sur la complexité croissante des options CLI** (https://danluu.com/cli-complexity/) - **Bash Cookbook** (Carl Albing, JP Vossen) - **Manuel GNU grep pour les expressions régulières** - **outils de test Regexer** (https://regexr.com/) - **Regex101** (https://regex101.com/) - **Documentation argparse** (https://docs.python.org/3/library/argparse.html) - **Tutoriels Matplotlib** (https://matplotlib.org/stable/tutorials)

Projet réalisé à Télécom Paris — 2025/2026
Équipe : Baptiste, Yoan, Théophile, Loïs
Encadrante : Xhevahire Tërnava